

CS 35L Software Construction

Lecture 5 – React, UI Development & Design Pattern Introduction

Tobias Dürschmid

Assistant Teaching Professor
Computer Science Department

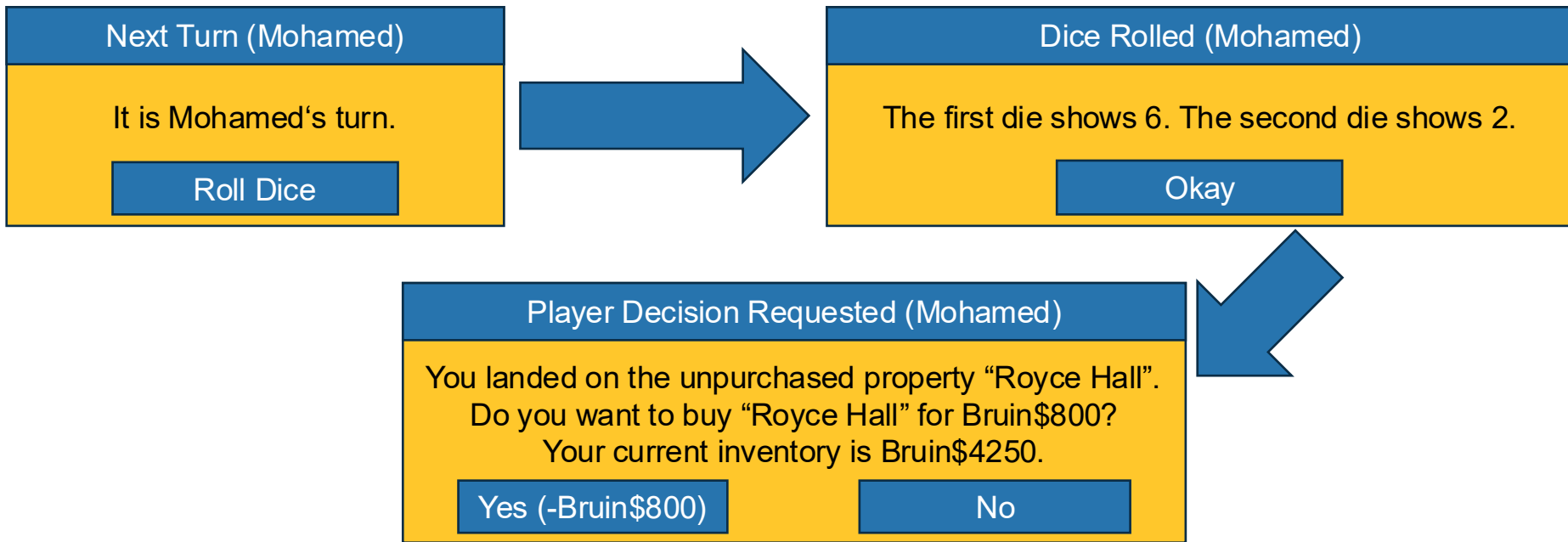


There are dozens of implementations of the game “Monopoly” with vastly different User Interfaces



Motivating Case Study: Digital Monopoly Game

There are dozens of implementations of the game “Monopoly” with vastly different User Interfaces. Even with just plain text UI!



What Should They All Have In Common?

All of them should implement
the exact same game logic!

So at least in theory they could all run
the exact same code just with a different UI
(ideally even with multiple UIs simultaneously)



Next Turn (Mohamed)

It is Mohamed's turn

Roll Dice

Dice Rolled (Mohamed)

The first dice shows 6.
The second dice shows 2.

Okay

Player Decision Requested (Mohamed)

You landed on the unpurchased property "Royce Hall".
Do you want to buy "Royce Hall" for Bruin\$800?
Your current inventory is Bruin\$4250.

Yes (-Bruin\$800)

No

Separation of Concerns

Systems should be **divided** into **distinct** sections, or **concerns**, where each section addresses a separate, specific goal, purpose, or responsibility.

The goal is to make the system **easier to develop, maintain, and evolve.**

Applies to **all software** development.
Not just UI

Important General Design Principle

Read more here: <https://www.geeksforgeeks.org/software-engineering/separation-of-concerns-soc/>

Front-End

Back-End



Separate the UI Implementation from the Domain Logic of your Application

Presentation Layer

Displays information to the user and collects **input** (e.g., positions of players on the board, style of the boards, buttons, ...)



Application Layer

Implements the **domain logic** and **behavior** (e.g., effects of community chest cards, player turns, interactions between players)

Doesn't have a clue that there even is a UI.

I mean who really needs a UI? Can't we just have command line interfaces with the shell???

Separate the UI Implementation from the Domain Logic of your Application

Presentation Layer



Application Layer

- Allows presentation layer to register a **callback** for state changes (e.g., `onBalanceChanged`)
- Allows presentation layer to retrieve information using **getter functions** (e.g., `getCurrentBalance()`)
- Allows presentation layer to forward user events (e.g., `buyProperty(name, user)`)

Doesn't have a clue that there even is a UI.

I mean who really needs a UI? Can't we just have command line interfaces with the shell???

The Observer Design Pattern

Pattern descriptions have many **different formats**. At minimum it should describe **problem, context, and solution**.

Context

Scenarios requiring **distributed event handling** systems or highly decoupled architectures like GUI, and signal processing

Problem

How can a **one-to-many dependency** between objects be maintained efficiently without making the objects tightly coupled?

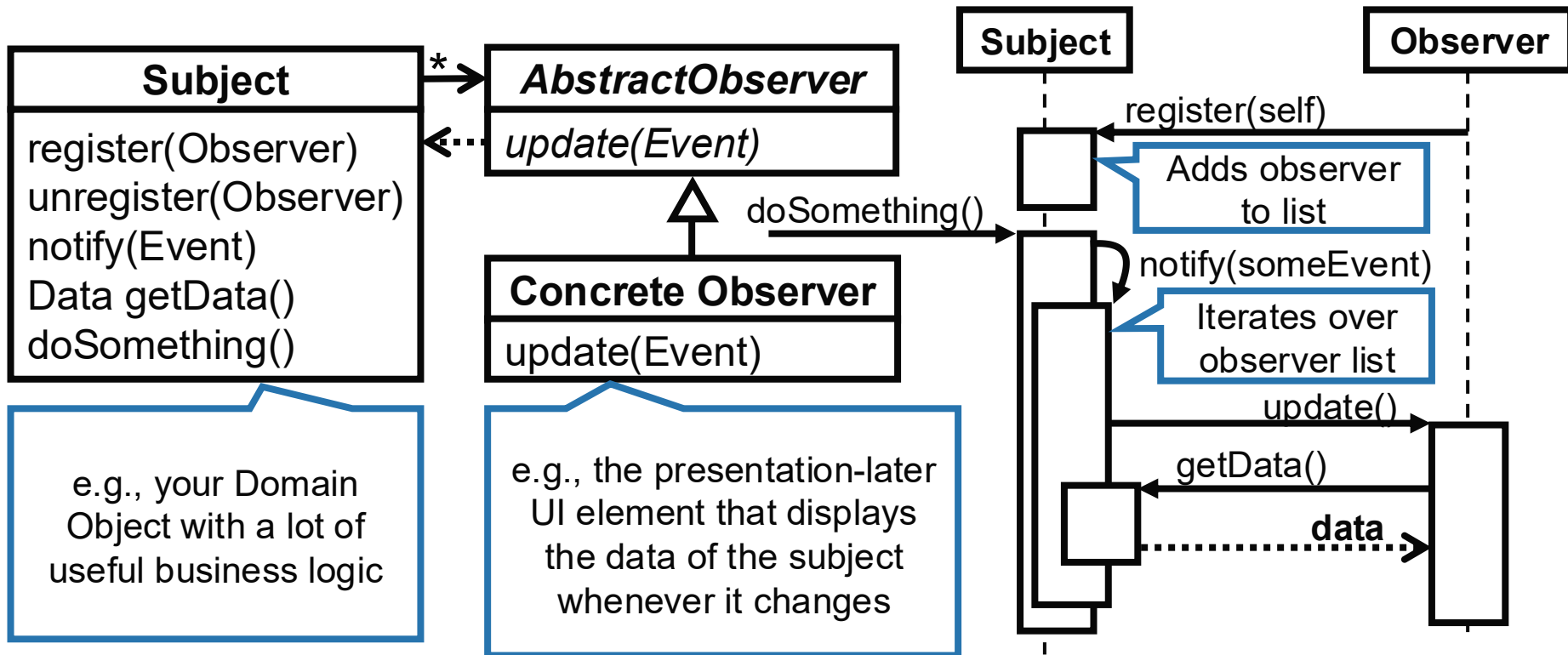
Solution

The pattern introduces two main roles: the ***Subject*** (the object sending updates after it has changed) and the ***Observer*** (the object listening to the updates of *Subjects*).

The Observer Design Pattern

Read more here:

<https://tobiasduerschmid.github.io/SEBook/designpatterns/observer.html>



Design Patterns

found in **many instances**

It is a **good** solution, but **not always the best** one

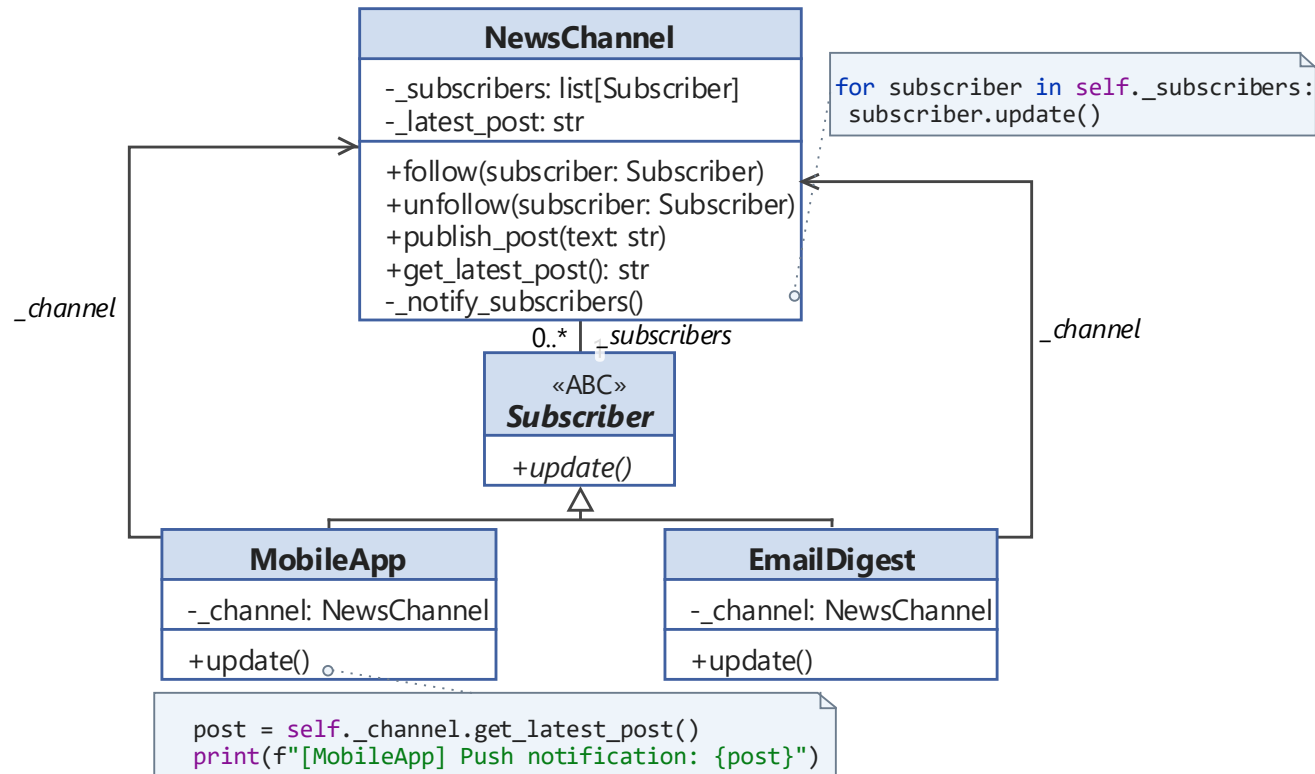
“A *pattern* is a common, acceptable **solution** to a reoccurring **problem** that arises in a specific **context**”

The problem is generic enough so that the it **generalizes** beyond a few concrete cases

Patterns always refer to a specific **situation, goal, or trade-off**. Patterns are **not universally** good

Design Patterns Add Roles To Classes / Objects

This is an Example of the Observer Design Pattern



Observer Pattern – Subscriber Interface

```
# =====  
# OBSERVER INTERFACE  
# =====  
class Subscriber(ABC):  
    """The Observer interface."""  
    @abstractmethod  
    def update(self):  
        pass
```

Observer Pattern – Subject (NewsChannel)

```
# SUBJECT
class NewsChannel:
    """The Subject that maintains a list of subscribers."""
    def __init__(self):
        self._subscribers: list[Subscriber] = []
        self._latest_post: str = ""

    def follow(self, subscriber: Subscriber):
        if subscriber not in self._subscribers:
            self._subscribers.append(subscriber)

    def unfollow(self, subscriber: Subscriber):
        self._subscribers.remove(subscriber)
```


Observer Pattern – Subject (NewsChannel)

```
# SUBJECT
class NewsChannel:

    [...]

    def publish_post(self, text: str):
        self._latest_post = text
        self._notify_subscribers()

    def get_latest_post(self) -> str:
        return self._latest_post

    def _notify_subscribers(self):
        for subscriber in self._subscribers:
            subscriber.update()
```

Observer Pattern – Concrete Observers

```
# CONCRETE OBSERVERS
class MobileApp(Subscriber):
    """A concrete observer that pulls state from the channel."""
    def __init__(self, channel: NewsChannel):
        self._channel = channel

    def update(self):
        post = self._channel.get_latest_post()
        print(f"[MobileApp] Push notification: {post}")

class EmailDigest(Subscriber):
    """Another concrete observer with different behavior."""
    def __init__(self, channel: NewsChannel):
        self._channel = channel

    def update(self):
        post = self._channel.get_latest_post()
        print(f"[EmailDigest] New email queued: {post}")
```

Observer Pattern – Client Code

```
# CLIENT CODE
channel = NewsChannel()

app = MobileApp(channel)
email = EmailDigest(channel)

channel.follow(app)
channel.follow(email)

channel.publish_post("New video uploaded!")
# [MobileApp] Push notification: New video uploaded!
# [EmailDigest] New email queued: New video uploaded!

channel.unfollow(email)

channel.publish_post("Live stream starting!")
# [MobileApp] Push notification: Live stream starting!
```

Lesson Learned:

Start By Considering Existing Solutions

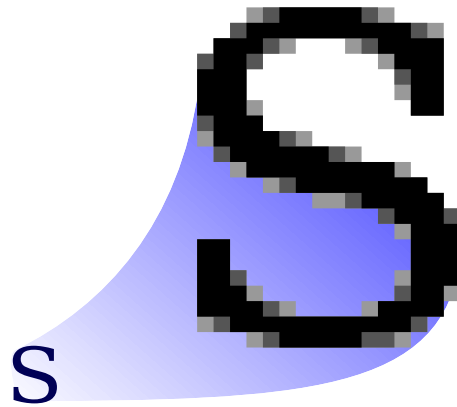
- Most problems have been **solved already** and described in a **well-documented** way
- **Knowing existing solutions and patterns in your field** can save you a lot of time and effort

Use SVGs Whenever You Can

Scalable Vector Graphics (SVG) are vector-based graphics defined by mathematical equations for shapes. They scale to any size and often have smaller file sizes.

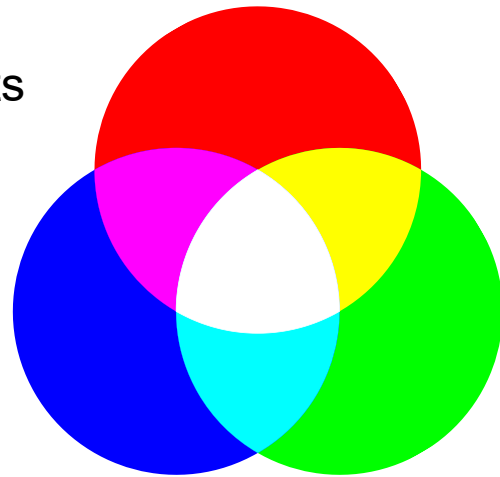


Bitmaps (e.g., JPEG, PNG, BMP, GIF, ...) are pixel-based graphics composed of a fixed grid of colored squares. Due to their general-purpose representation, they are supported by more editing software.



There are many Different Color Representations

- **RGB** (**Red**, **Green**, **Blue**) is the most common format to represent digital colors. The resulting color is a mix of red, green, and blue light.
 - The Hex-Color representation uses two hexadecimal digits for each color (**#RRGGBB** e.g., **#2774AE**, **#FFD100**)
- **HSL/BSH** (Hue, Saturation, Luminance/Brightness) lets you represent colors based on their color tone (hue), and color intensity (saturation) and brightness (luminance)
- **CMYK** (**Cyan**, **Magenta**, **Yellow**, **Keycolor** (Black)) represents colors based on the subtractive color model used in printing and is less common for screen-based rendering



Important HTML Tag

Content Tags

- **Headings** (<h1> to <h6>)
e.g., <h1>Top Heading</h1>
- **Paragraphs** (<p>) e.g., <p>Example</p>
- **Lists** (unordered list
and ordered lists)
e.g.,
 first item
 second item

Top Heading

Example

1. first item
 2. second item
- First item
 - Second item

Important HTML Tag

Text Formatting Tags

- **Bold** (previously: or)
e.g., Important Text
- **Italics** (previously: <i> or <I>)
e.g., emphasis
- **Underline** (<ins> previously: <U>)
e.g., <ins>inserted</ins>
- **Strikethrough** ()
e.g., deleted

Important Text

emphasis

inserted

~~deleted~~

Important HTML Tag

Text Formatting Tags

- **Button** (`<button>`)
e.g., `<button>Click me</button>`
- **Links** (`<a>`)
e.g., `<a href="ucl.edu">anchor</a>`
- **Images** (`<img>`)
e.g., ``
- **Inputs** (`<input>` with various types)
e.g., `<input type="checkbox">check me</input>` (or text, file, email, radio, ...)

Click me

[anchor](#)

Ucla

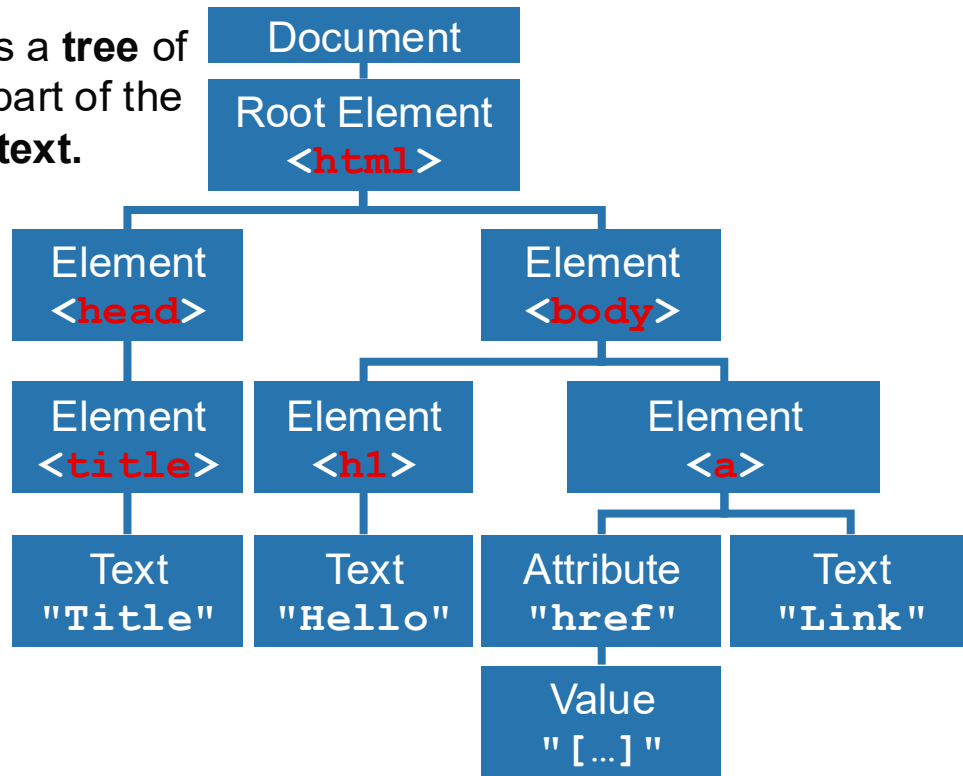


Check me

HTML DOM (Document Object Model) is a programming interface for HTML documents.

It represents the structure of an HTML page as a **tree** of objects, where each **object** corresponds to a part of the document, such as an **element**, **attribute**, or **text**.

```
<html>
<head>
  <title>My Title</title>
</head>
<body>
  <h1>Hello</h1>
  <a href=" [...]">Link</a>
</body>
</html>
```



CSS (Cascading Style Sheets)

- **Declarative-style** computer programming language used to **define the visual presentation and layout** of web pages.
- Works in conjunction with markup languages like **HTML** (which **provide the structure and content of a webpage**)

CSS
Visual presentation & Layout

HTML
Content & Structure

**Separation of
Concerns** 

More details here:

<https://www.w3schools.com/css/>

CSS can define style classes for HTML divs (divisions, used as containers)

example.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Example</title>
  <link rel="stylesheet"
        href="example.css">
</head>
<body>
  <h1>CSS Example</h1>
  <div class="rounded-box">
    <p>This will be styled
      with CSS.</p>
  </div>
</body>
</html>
```

example.css

```
h1 { /* Heading 1 will be navy colored
and underlined */
  color: navy;
  text-decoration: underline;
}

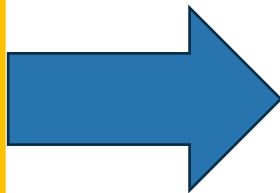
.rounded-box {/* Will become a class
               for divs*/

  width: 200px;
  background-color: #FFD100;
  border: 5px solid #2774AE;
  border-radius: 10px; /* corners */
  text-align: center;
  font-family: sans-serif;
}
```

CSS can define style classes for HTML divs (divisions, used as containers)

example.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Example</title>
  <link rel="stylesheet"
        href="example.css">
</head>
<body>
  <h1>CSS Example</h1>
  <div class="rounded-box">
    <p>This will be styled
      with CSS.</p>
  </div>
</body>
</html>
```



CSS Example

This paragraph will be styled with CSS.

React.js (aka. React, ReactJS)

- React is a free and open-source front-end JavaScript library for building user interfaces
- maintained by Meta (formerly Facebook)
- React has a **declarative nature**
 - you describe ***what*** you want the UI to look like for a given state
 - React handles the steps to achieve that UI and **keep it updated efficiently when the state changes**.
 - In contrast, **imperative** programming focuses on describing *how* to achieve a result by manually manipulating the UI elements

React uses JSX (JavaScript XML)

- JSX is a **syntax extension** that allows developers to **write HTML-like code within JavaScript**. This makes it easier to define the structure and content of UI components.

```
React.createElement('h1', null, 'Hello, ', name, '!');
```

```
const name = "CS 35L"  
[...]
```

JavaScript constant (or variable)
defined somewhere

```
return <h1>Hello, {name}!</h1>;
```

Use JavaScript within HTML-like
tags via curly braces.



JSX — JavaScript XML

React and JSX

React is a free, open-source front-end JavaScript library maintained by Meta (formerly Facebook) for building user interfaces.

React has a **declarative nature**: you describe *what* you want the UI to look like, and React handles the steps to achieve that UI and keep it updated efficiently when state changes. This is in contrast to **imperative programming** (plain

step1/App.jsx

```
      {propertyName} and ${price}
8
9  function App() {
10     return (
11       <div style={{ padding: '16px', fontFamily: "'Georgia',
12         serif" }}>
13         <h1>Welcome to UCLA Monopoly!</h1>
14         <p>Most expensive property: Royce Hall</p>
15         <p>Price: $1000</p>
16       </div>
17     );
18   }
19 }
```

● LIVE PREVIEW

↶ Refresh

Welcome to UCLA Monopoly!

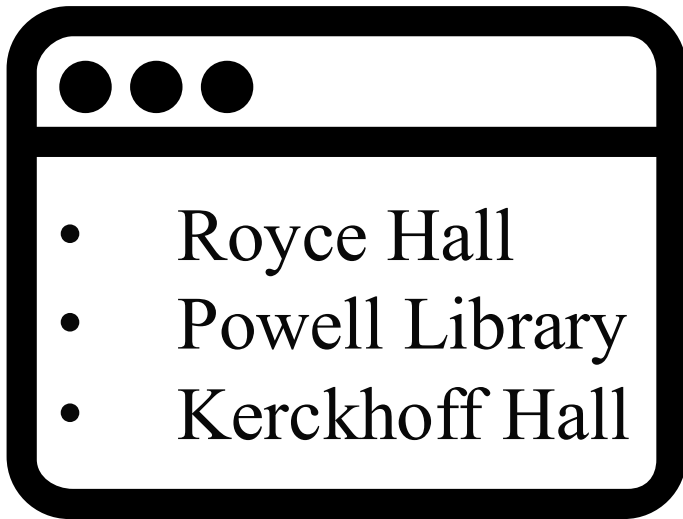
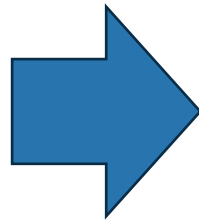
Most expensive property: Royce Hall

Price: \$1000

Map function

```
const names = ["Royce Hall", "Powell Library",  
"Kerckhoff Hall"]  
  
[...]
```

```
return (<ul>  
  {names.map((name, index) => (  
    <li key={index}>{name}</li>  
  ))}  
</ul>);
```



The Map Function — Rendering Lists

Rendering Lists with

`.map()`

In React, you render lists by using JavaScript's built-in `.map()` function directly inside JSX.

```
const properties = [  
  { name: "Royce Hall",  
    price: 1000 },  
  { name: "Powell Libra  
ry", price: 500 },  
];  
// [...]  
return (<ul>
```

step2/App.jsx

```
8      return  
9      <div style={{ padding: '16px', fontFamily: "'Georgia',  
10         serif" }}>  
11         <h2>UCLA Monopoly – Property Listing</h2>  
12         <ul>  
13           {/* TODO: Use properties.map() to render each  
14             property */}  
15           {/* Each <li> should show: name – $price */}  
16           {/* Don't forget the key prop! */}  
17         </ul>  
18       </div>
```

LIVE PREVIEW

Refresh

UCLA Monopoly — Property Listing

React Applications use *Components*

- Components are self-contained, reusable pieces of UI that manage their own rendering logic.
- This modularity promotes code **reusability**, **simplifies development**, and **improves maintainability**.
- React Component are **Defined as Functions** with a **Single Return Element**
 - Inputs to the functions are called **props**

Can be referenced via:
`<WelcomeMessage
name="CS35L" />`

Component Parameters

Component Parameter
Reference

```
function WelcomeMessage(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
export default WelcomeMessage;
```

Makes the component visible to outside modules

The Problem: Hard-Coded Cards

The current code has a single hard-coded card for Royce Hall. Want three cards? You'd have to copy-paste and change every value by hand.

The Solution: A Reusable Component with Props

Props (short for *properties*) are read-only inputs passed to a component — just like function arguments. Extract the card into a **component** and make the varying parts props:

```
const MonopolyProperty =  
({ name, price, colorGroup
```

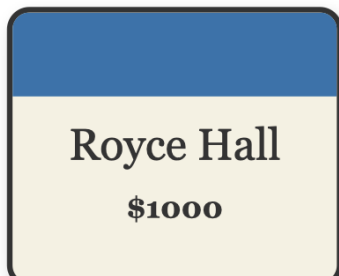
step3/styles.js

step3/App.jsx

```
1 // This card is hard-coded for Royce Hall – NOT reusable!  
2 // TODO: Extract a MonopolyProperty component with props:  
3 //     name, price, colorGroup  
4 // TODO: Render three property cards with different data  
5 function App() {  
6   return (  
7     <div style={{ display: 'flex', flexWrap: 'wrap', padding: '16px' }}>  
8       <div style={cardStyle}>  
9         <div style={getHeaderStyle("ucla-blue")}></div>  
10        <div style={bodyStyle}>  
11          <h4>Royce Hall</h4>  
12          <p><strong>$1000</strong></p>  
13        </div>  
14      </div>  
15    )  
16  }
```

LIVE PREVIEW

Refresh



React Components manage their own *State*

```
import React, { useState } from 'react';  
function Counter(props) {  
  const [count, setCount] = useState(props.initialCount);
```

Initial value of the state

State variable

State change function name

```
    const increment = () => {  
      setCount(count + 1);  
    };  
    return (  
      <>  
        <p>Current Count: {count}</p>  
        <button onClick={increment}>  
          Increment  
        </button>  
      </>  
    );  
  } export default Counter;
```

Reference to the state variable. Changes whenever the state variable is changed via the state change function

Click event handler (the name of the function to be called when the user clicks on the button)

Makes the component visible to outside modules

React Components manage their own *State*

```
import React, { useState } from 'react';
```

Unwrap props

Initial value of the state

```
function Counter({initialCount}) {
```

```
  const [count, setCount] = useState(initialCount);
```

State variable

State change function name

```
  const increment = () => {
```

```
    setCount(count + 1);
```

```
  };
```

```
  return (
```

```
    <>
```

```
      <p>Current onClick={increment}>
```

```
        Increment
```

```
      </button>
```

```
    </>
```

```
  );
```

```
  Count: {count}</p>
```

```
  <button> export default Counter;
```

Reference to the state variable. Changes whenever the state variable is changed via the state change function

Click event handler (the name of the function to be called when the user clicks on the button)

Makes the component visible to outside modules

React Components manage their own *State*

```
import React, { useState } from 'react';  
function Counter({initialCount, countName}) {  
  const [count, setCount] = useState(initialCount);
```

You can add props

Initial value of the state

State variable

State change function name

```
    const increment = () => {  
      setCount(count + 1);  
    };  
    return (  
      <>  
        <p>{countName}: {count}</p>  
        <button onClick={increment}>  
          Increment  
        </button>  
      </>  
    );  
  }  
  export default Counter;
```

Reference to the state variable. Changes whenever the state variable is changed via the state change function

Click event handler (the name of the function to be called when the user clicks on the button)

Makes the component visible to outside modules



Solution Explanation

We added `const [houses, setHouses] = useState(0)` to track the number of houses. The `buyHouse` handler increments houses up to `MAX_HOUSES + 1` (5 total — triggering the hotel upgrade).

`renderBuildings(houses)` in `App.jsx` converts the count into visual icons. The button is disabled once fully upgraded, and each card maintains its own independent state.

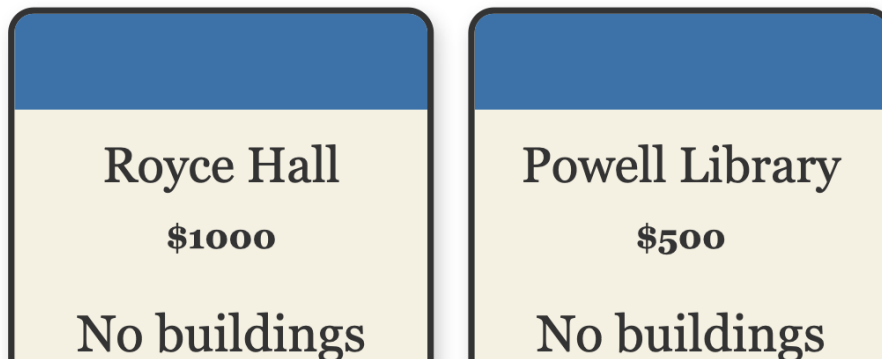
step4/styles.js

step4/App.jsx

```
1  const { useState } = React;
2
3  const MAX_HOUSES = 4;
4
5  // Converts house count to visual building icons
6  function renderBuildings(houses) {
7    if (houses === 0) return "No buildings";
8    if (houses > MAX_HOUSES) return HOTEL_EMOJI;
```

● LIVE PREVIEW

↶ Refresh



React employs a *Virtual DOM*

- Virtual DOM is a **lightweight representation** of the actual DOM.
- When data changes, React **first updates the Virtual DOM**, then efficiently **calculates the differences** with the real DOM and **applies only the necessary changes** to the real DOM.
- Minimizes direct DOM manipulation and **improves performance**

Virtual DOM — Complete UCLA Monopoly

The Virtual DOM

React employs a **Virtual DOM** — a lightweight JavaScript representation of the actual DOM.

When data changes, React:

1. First updates the **Virtual DOM** (fast — it is just a JavaScript object tree)
2. Calculates the differences between the new and previous Virtual

step5/styles.js

step5/monopoly-game.js

step5/PropertyCard.jsx

step5/App.

```
12   if (result) {
13     setWallet(result.wallet);
14     setProperties(result.properties);
15   }
16 };
17
18 const totalRent = calculateTotalRent(properties);
19
20 const collectRent = () => {
21   // TODO: Add totalRent to the wallet using setWallet
22 };
```

● LIVE PREVIEW

Refresh

UCLA Monopoly

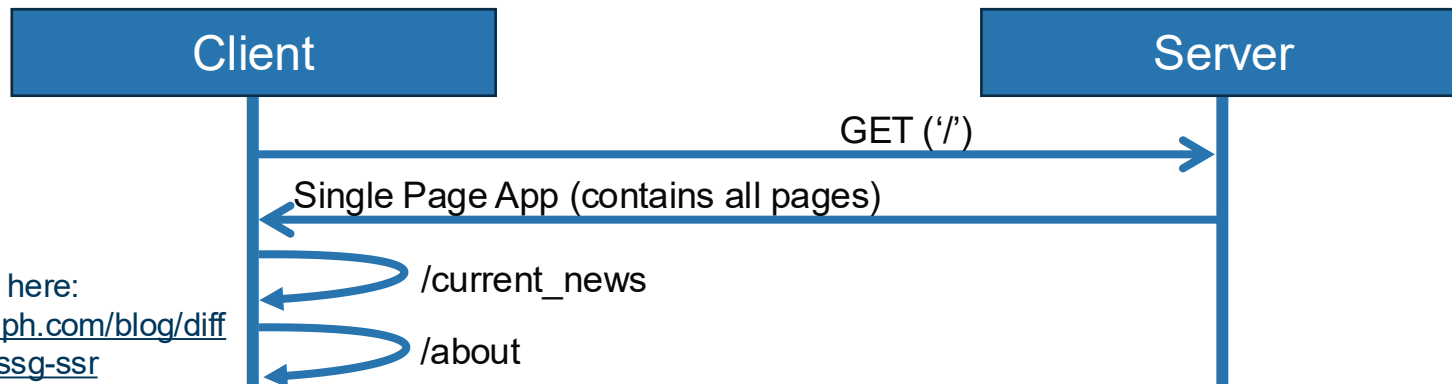
Wallet: \$1500 Properties: 0/6 Rent/turn: \$0

Collect Rent

Single Page Application (SPA)

Most common in React

- SPAs are structured as a **single HTML page** that has no preloaded content.
- **Content is loaded dynamically via JavaScript** for the entire application and housed within a single HTML page.
- The JavaScript code houses all the data relating to the application logic, UI, and communication with the server

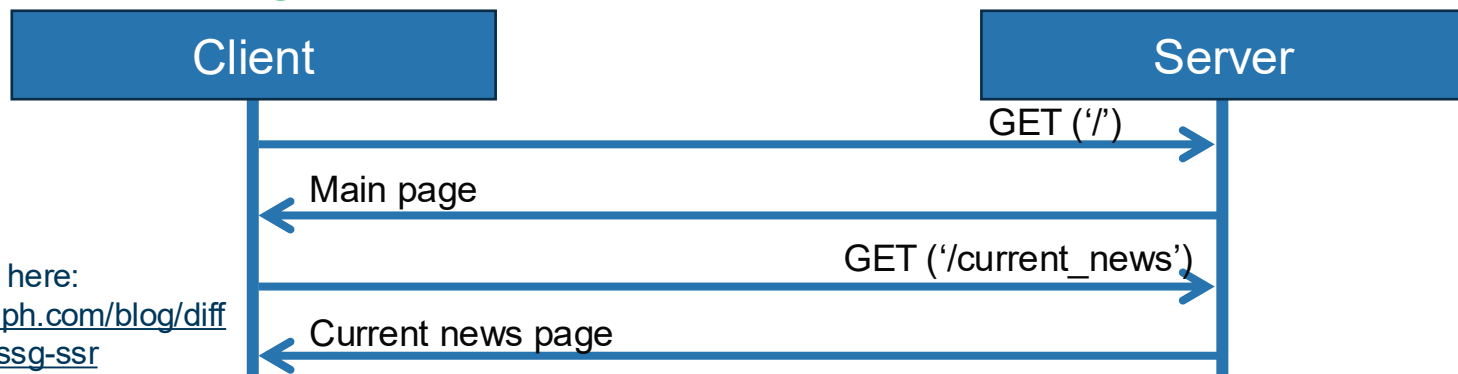


More details here:
<https://hygraph.com/blog/difference-spa-ssg-ssr>

Server-Side Rendering (SSR)

Frameworks like Next.js
(which use React) do this

- Clients receive a fully rendered page (rendered on the server)
- Requires more **server compute**
- Faster **initial load time** (since only the current page is sent)
- Longer response time for **user interaction**
- Better search **engine optimization**



More details here:
<https://hygraph.com/blog/difference-spa-ssg-ssr>

Please Fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

Please summarize **three insights** you learned about UI Development, Design Patterns, or React today.

Question 2

1 pts

Please explain *how* using the **Observer Design Pattern** for UI elements supports the design principle of **Separation of Concerns**

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images generated with Gemini, from Flaticon.com (Creators: Freepik) and www.svgrepo.com (Creators: Solar Icons, Iconsax, Giovana, Esri)